Planejamento

- **Introdução**
- **Planejamento x Resolução de problemas**
- **Abordagens**
  - Cálculo situacional + provador de teoremas
  - STRIPS + POP
- **Limitações e desenvolvimentos**
- **Conclusões**

Conceitos básicos

- **Plano: seqüência ordenada de ações**
  - problema: obter banana, leite e uma furadeira
  - plano: ir ao supermercado, ir à seção de frutas, pegar as bananas, ir à seção de leite, pegar uma caixa de leite, ir ao caixa, pagar tudo, ir a uma loja de ferramentas, ..., voltar para casa.

- **O Agente Planejador**
  - combina **agente baseado em conhecimento** com o **agente resolvedor de problemas** (planejar adiante)
    ≠ *representação de estados, objetivos e ações*
    ≠ *representação e construção da solução*

Agente Planejador Simples

- **3 fases:** Percepção do ambiente; Planejamento (tempo ilimitado) e Execução do plano (passo a passo)

- **Algoritmo**

Function Simple-planning-agent (*percept*) **returns** *action*
t := 0
Tell (KB, Make-percept-sentence (*percept, t*)
current := State-description (*KB,t*)
If *p* = NoPLan **then**
  G := Ask(KB, Make-a-goal-query(*t*))
  p := Ideal-planner(*current, G, KB*)
If *p* = NoPlan or p is empty **then** *action* := NoOp
else action := First (*p*)
    p := Rest (*p*)
Tell (KB, Make-action-sentence (*action,t*))
t := t + 1
return *action*

Resol. de problemas x planejamento

- **Representação em RP**
  - **Ações** : programas que geram o estado sucessor
  - **Estados** : descrição completa
    - *problemático em ambientes inacessíveis*
  - **Objetivos**: função de teste e heurística
  - **Planos**: totalmente ordenados e criados incrementalmente a partir do estado inicial
    - *Ex. posições das peças de um jogo*

- **Exemplo do supermercado**
  - **estado inicial:** em casa, sem objetos desejados
  - **estado final**: em casa com objetos desejados
  - **operadores**: tudo o que o agente pode fazer
  - **heurística**: número de objetos ainda não possuídos

Exemplo em RP



Limitações da RP

- **Fator de ramificação grande**
- **A função heurística apenas escolhe o estado mais próximo do objetivo. Não permite descartar ações a priori (≈ adivinhação)**
- **Não permite abstração dos estados parciais**
- **Considera ações a partir do estado inicial, uma após a outra**

Planejamento: 3 idéias principais (1/2)

- **Representação dos estados, objetivos e ações usando lógica (descrições parciais dos estados)**
  - pode conectar diretamente estados (sentenças) e ações (pré-condições + efeitos)
  - ex. estado: Have (Milk), ação: Buy(milk)=> Have(Milk)
- **Liberdade de adicionar ações ao plano quando forem necessárias**
  - ordem de planejamento ≠ ordem de execução
  - primeiro, o que é importante : Buy(Milk)
  - diminui fator de ramificação

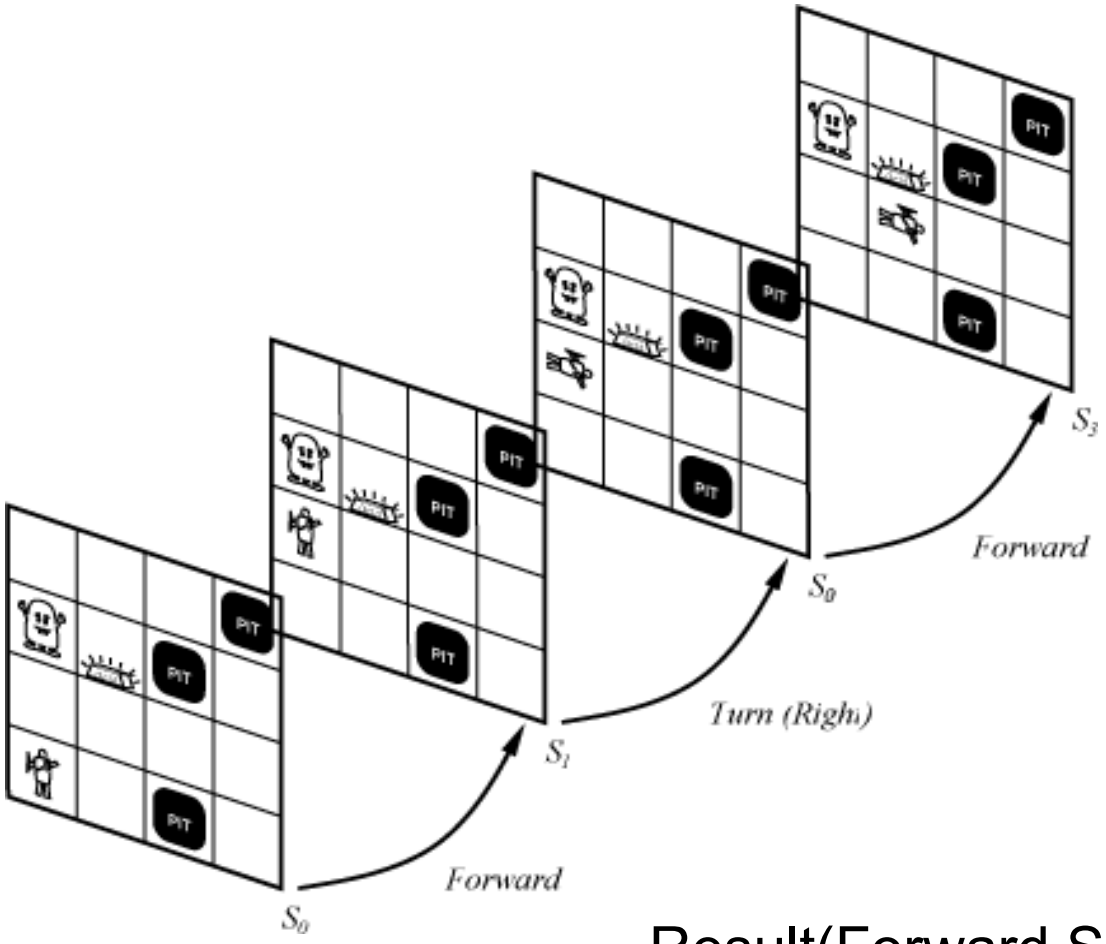
Planejamento: 3 idéias principais (2/2)

- **Aproveitar a independência entre a maioria das partes do mundo (sub-goaling)**
  - sub-plano supermercado, sub-plano loja de ferramentas
  - Não funciona para Puzzles!!!

Relembrando o Cálculo Situacional

- **O mundo consiste em uma seqüência de situações**
  - situação N ===**ação**===> situação N+1
- **Predicados que <u>mudam</u> com o tempo têm um argumento de <u>situação</u> adicional**
  - Ao invés de Em*(*Agente*,*local*)* *teremos* *(*Em*(*Agente,[1,1],S0*)* $\wedge$ Em*(*Agente,[1,2],S1*))*
- **Predicados que denotam que não mudam com o tempo**
  - não necessitam de argumentos de situação
  - ex. no mundo do Wumpus:*Parede*(*0,1*) e *Parede*(*1,0*)
- **Para representar as mudanças no mundo: função *Resultado***
  - *Resultado* (*ação,situação N*) = situação N+1

Relembrando o Cálculo Situacional



Result(Forward,S_0) = S_1

Result(Turn(Right),S_1) = S_2

Result(Forward,S_2) = S_3

Axiomas estado-sucessor

- **As <u>ações</u> são descritas pelos seus <u>efeitos</u>**
  - especificando-se as propriedades da situação resultante da realização da ação
  - por meio dos **axiomas de efeito** (o que muda) e dos **axiomas frames (o que não muda)**

- **Axioma de efeito + axioma de frames = axioma estado-sucessor**
  - *uma coisa é verdade depois* $\Leftrightarrow$ [*uma ação acabou de torná-la verdade* $\vee$ *ela já era verdade e nenhuma ação a tornou falsa*]

  - Ex. $\forall$ *a,x,s* *Segurando*(*x, Resultado*(*a,s*)) $\Leftrightarrow$ [$\Box$ (*a* = *Pegar* $\wedge$ (*Presente* (*x, s*) $\wedge$ *Portável*(*x*)) $\vee$ (*Segurando* (*x,s*) $\wedge$ (*a* $\neq$ *Soltar*)]

Planejando com Calculo de Situações

- **Estado inicial: sentença lógica**
  At(Home, S0) $\wedge \neg$ Have(Milk , S0) $\wedge \neg$ Have(Bananas , S0) $\wedge \neg$ Have(Drill , S0)

- **Estado Objetivo: pergunta lógica** (p/ unificação)
  At(Home, S) $\wedge$ Have(Milk , S) $\wedge$ Have(Bananas , S) $\wedge$ Have(Drill , S)

- **Operadores: conjunto de axiomas de estado sucessor**
  $\forall$ a,s Have(Milk, Result(a, s)) $\Leftrightarrow$
  [(a = Buy(Milk) $\wedge$ At(supermarket, s)
  $\vee$ (Have(Milk, s) $\wedge \alpha \neq$ Drop(Milk))]

- **Notação**
  - Result(a,s) - uma ação
  - Result'(p,s) - seqüência de ações

Planejando com Calculo de Situações

- **Estado Objetivo: pergunta lógica**

  At(Home,Result'(p, S0)) $\wedge$ Have(Milk, Result'(p, S0)) $\wedge$
  Have(Bananas, Result'(p, S0)) $\wedge$ Have(Drill, Result'(p, S0))

- **Solução:**

  p = [Go(SuperMarket), Buy(Milk), Buy(Bananas),
  Go(HardwareStore), Buy(Drill), Go(home)]

- **Limitações**
  - Eficiência da inferência em lógica de primeira ordem
  - Nenhuma garantia sobre a qualidade da solução
    - *ex. pode haver passos redundantes*

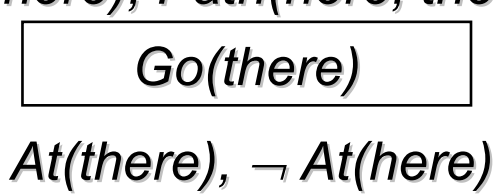
Solução: especializar linguagem (STRIPS) e algoritmo (POP)

- **Estados:** conjunção de literais sem variáveis
  - *Inicial: At(Home)*
  - *Hipótese do mundo fechado: ¬ Have(Milk) ^ ¬ Have(Bananas)*
  - *Final: At(Home) ^ Have(Milk) ^ Have(Bananas) ^ Have(Drill)*
- **Objetivos:** conjunção de literais e possivelmente variáveis (∃)
  - *At(Home) ^ Have(Milk) ^ Have(Bananas) ^ Have(Drill)*
  - *At(x)^ Sells(x, Milk)*
- **Ações:**
  - Descritor da **ação**: predicado lógico
  - **Pré-condição:** conjunção de literais positivos
  - **Efeito**: conjunção de literais (positivos ou negativos)

Exemplo de ações em STRIPS
(STanford Research Institute Problem Solver)

- **Operador para ir de um lugar para outro**
  - *Op(ACTION: Go(there),*
    PRECOND:At(here) ^ Path(here, there),
    EFFECT:At(there) ^ ¬ At(here))

- **Notação alternativa**



- **Observações**
  - Não há variáveis de situação (S) explícitas
  - **Esquema de operador** = classe de ação, predicado com variáveis não instanciadas
  - **Operador** = instanciação particular, predicado sem variáveis
  - O **operador O é aplicável** a um estado S, se precond (O) $\subset$ s

Tipos de Planejadores

- **Controle**
  - Progressivo: estado inicial -> objetivo
  - Regressivo: objetivo -> estado inicial
    - *mais eficiente (há menos caminhos partindo do objetivo do que do estado inicial)*
    - *problemático se existem múltiplos objetivos*

- **Espaços de busca**
  - Espaço de situações (como na resolução de problemas)
  - Espaço de planos (planos parciais)
    - *mais flexível*
    - *evita engajamento prematuro*

Busca no espaço de planos

- **Idéia**
  - Buscar um plano desejado em vez de uma situação desejada (espécie de meta-busca)
  - parte-se de um plano inicial (parcial), e aplica-se 2 tipos de operadores até chegar a um plano final (completo)
- **Plano inicial**
  - passos Start e Finish
- **Plano final**
  - **Completo** - toda a pré-condição de todo passo é alcançada por algum outro passo
  - **Consistente** - não há contradições
    - *nos ordenamentos das ações*
    - *nas atribuição de variáveis*

Busca no espaço de planos: operadores

- **Operador tipo 1: operador de refinamento**
  - adiciona restrições ao plano
  - tipos de restrições
    - *adicionar um passo*
    - *instanciar variável*
    - *ordenar passos*

- **Operador tipo 2: operador de modificação**
  - plano parcial representa conjunto potencial de planos completos totalmente definidos
  - operadores de refinamento eliminam planos potenciais, enquanto os de modificação adicionam novos planos
    - *operadores de revisão (para corrigir planos)*
    - *operadores de decomposição hierárquica*

Exemplo informal: colocar meias e sapatos

- **Plano inicial**
  - start
  - end (pré-condição: estar com meias e sapatos)
- **Operadores**
  - calçar meia direita (pré-condição: pé direito descalço; efeito: pé direito com meia)
  - calçar sapato direito (pré-condição: pé direito com meia; efeito: pé direto com meia e sapato)
  - calçar meia esquerda...
  - calçar sapato esquerdo...
- **Plano final?**
  - Existem vários possíveis....
  - Como representar isto?

Representações de planos: Linguagem

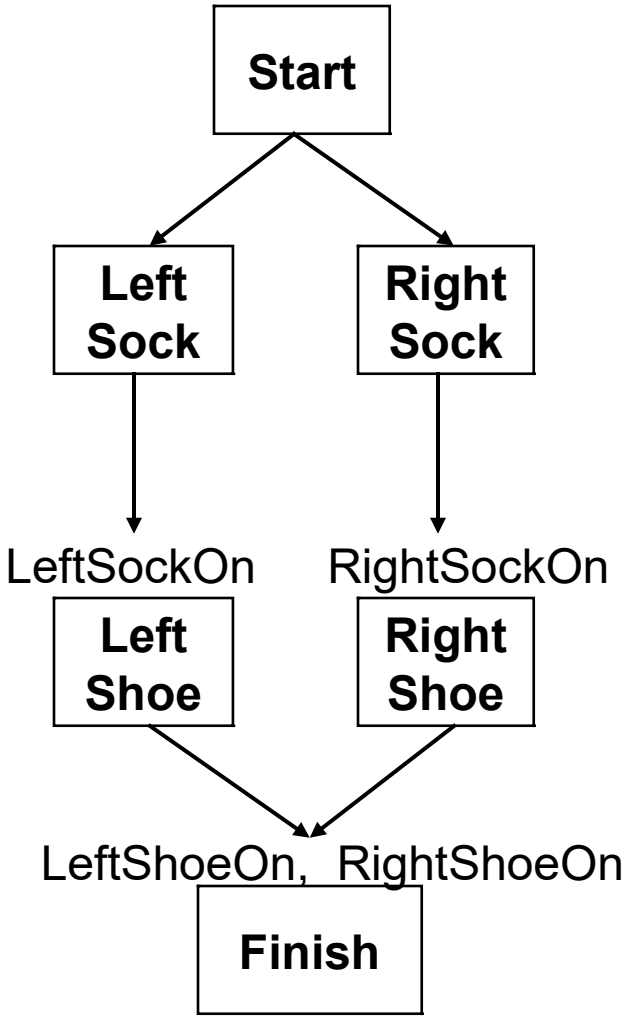
- **Passos = {S1: operador1, ..., Sn: operadorN},**
- **Ordem = { S1 < Sk < Sn },**
  - o que não significa que entre S1 e Sk não exista outro passo
- **Ligações causais = {Si** $\xrightarrow{c}$ **Sj}**
  - efeitos Si = pré-condições de Sj
- **Bindings = { var = constante, var1 = var2},**
- **Operador = Op(Ação(x), Precond(y), Efeito(z))**

Devolta às meias e sapatos

- **Objetivo: RightShoeOn ^ LeftShoeOn**
- **Operadores**
  - *–Op(ACTION:RightShoe , PRECOND: RightSockOn , EFFECT: RightShoeOn)*
  - *–Op(ACTION: RightSock , EFFECT: RightSockOn)*
  - *–Op(ACTION:LeftShoe , PRECOND: LeftSockOn , EFFECT: LeftShoeOn)*
  - *–Op(ACTION: LeftSock , EFFECT: LeftSockOn)*
- **Plano inicial**

Plan(STEPS:{S1: Op(ACTION: Start),
S2: Op(ACTION: Finish,
PRECOND: RightShoeOn ^ LeftShoeOn)},
ORDERINGS: { S1 < S2 },
BINDINGS: {},
LINKS: {})

Plano (de ordem) parcial

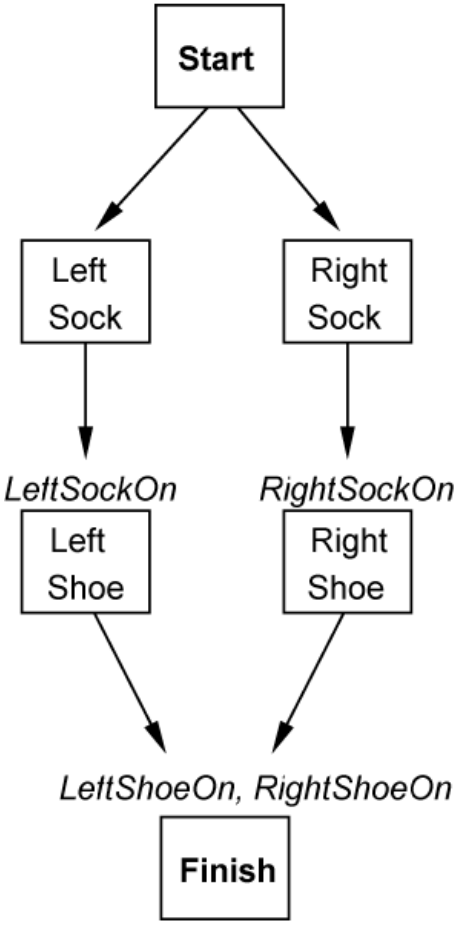


Plano final: características

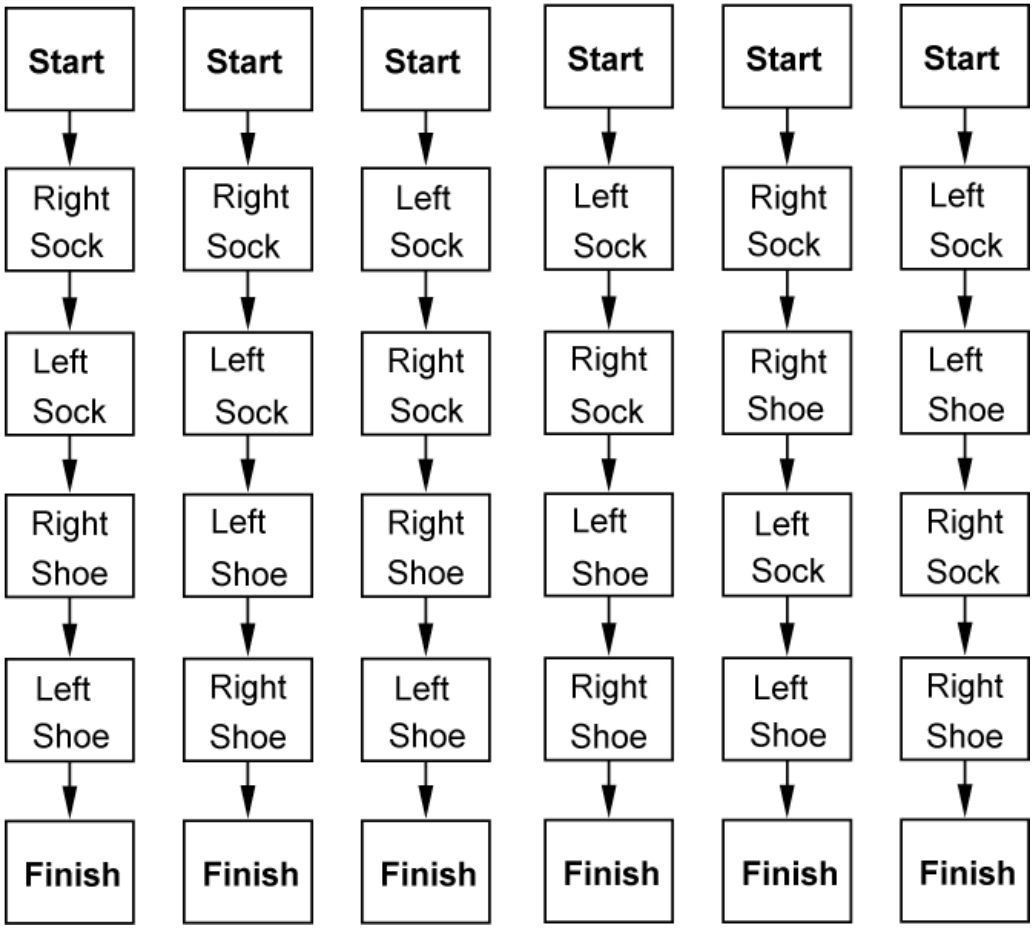
- **Plano final**
  - **Completo** - toda a pré-condição de todo passo é alcançada por algum outro passo
  - **Consistente** - não há contradições nos ordenamentos ou nas atribuição de variáveis
  - mas não necessariamente totalmente ordenado e instanciado
- **Ordem total x Ordem parcial**
  - lista simples com todos os passos um atrás do outro
  - **Linearizar** um plano é colocá-lo na forma “ordem total”
- **Instanciação completa** de um plano
  - todas variáveis são instanciadas
  - ex. posso decidir que vou a um supermercado sem dizer qual...

Linearização do exemplo dos sapatos

Partial Order Plan:



Total Order Plans:



Princípio do menor engajamento

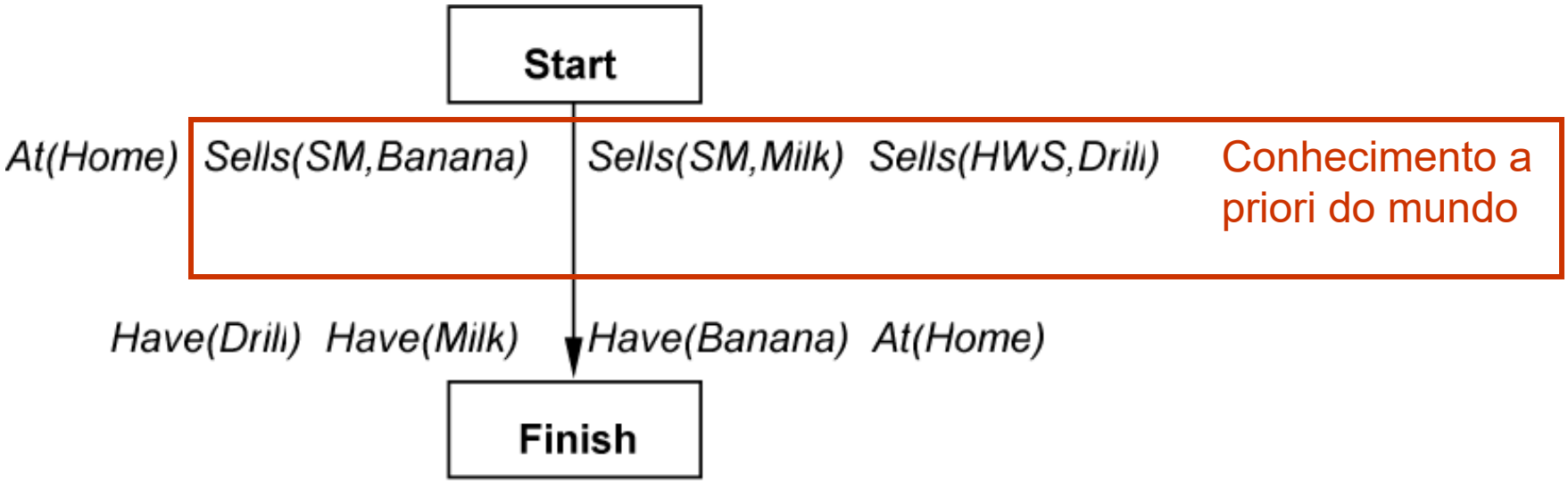
- **Para que então deixar o plano não totalmente ordenado e instanciado?**
- **Princípio do menor engajamento** (least commitment planning)
  - não faça hoje o que você pode fazer amanhã
  - ordem e instanciação parcial são decididas quando necessário
  - evita-se backtracking!
- **Exemplo**
  - para objetivo have(Milk), a ação Buy(item, store), se instancia só item => Buy (Milk,store)
  - para as meias/sapatos: botar cada meia antes do sapato, sem dizer por onde começa(esq/dir)

POP (Partial Order Planning)

- **Existindo a linguagem (STRIPS), falta o algoritmo...**
- **Características do POP**
  - Algoritmo não determinístico
  - A inserção de um passo só é considerada se atender uma precondição não atingida
  - Planejador regressivo
  - É correto e completo, assumindo busca em largura ou em profundidade iterativa
- **Idéia do algoritmo**
  - identifica passo com pré-condição não satisfeita
  - introduz passo cujo efeito é satisfazer esta pré-condição
  - instancia variáveis e atualiza os links causais
  - verifica se há ameaças e corrige o plano se for o caso

Voltando ao exemplo das compras...

- **Plano inicial**

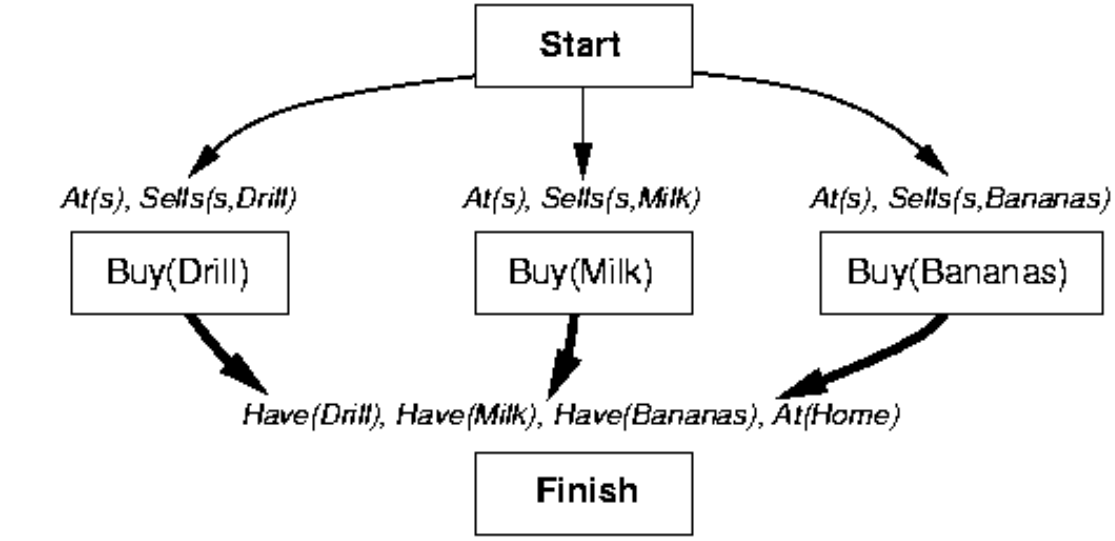


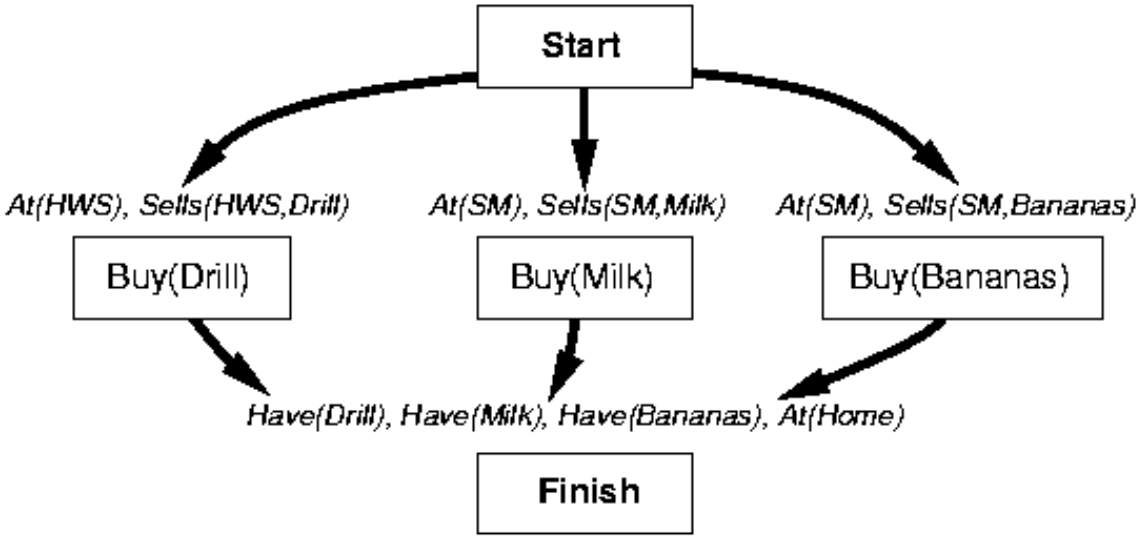
- **Ações**

  Op(ACTION: **Go**(there), PRECOND: At(here),
  EFFECT: At(there) $\wedge$ $\neg$ At(here))

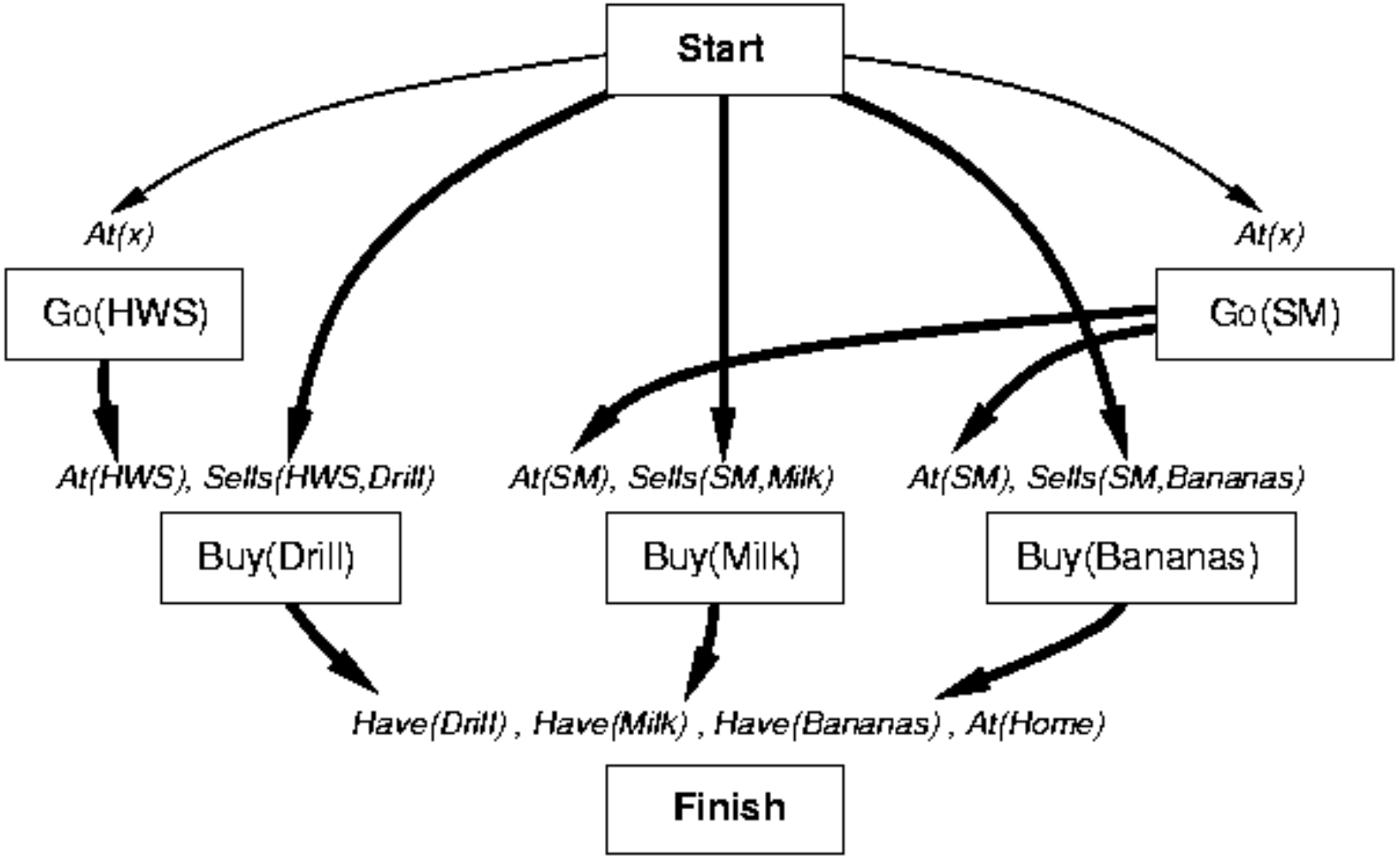
  Op(ACTION: **Buy**(x), PRECOND: At(store) ^ Sells(store, x),
  EFFECT: Have(x))

Planejamento Parcial - Exemplo

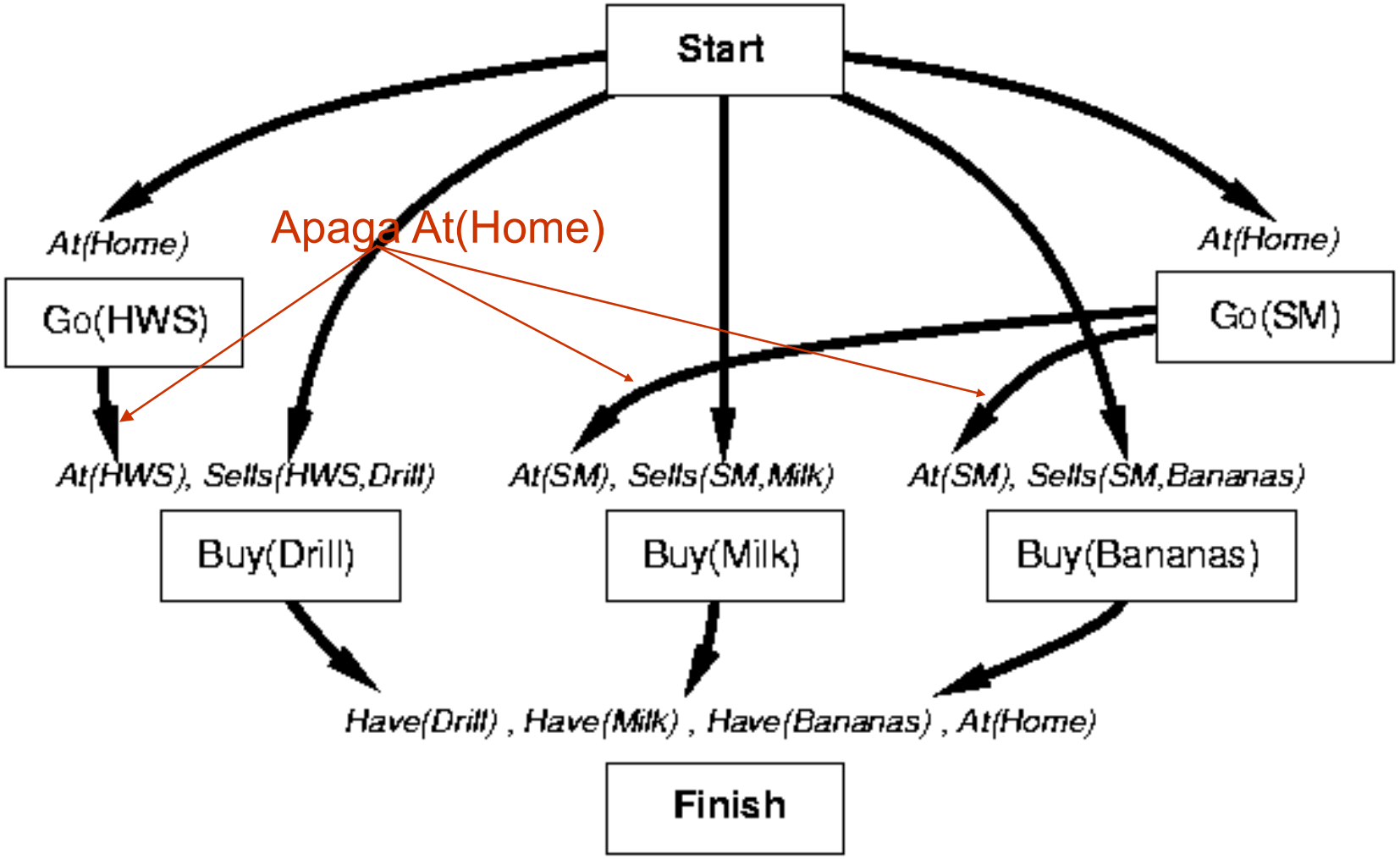




Planejamento Parcial - Exemplo



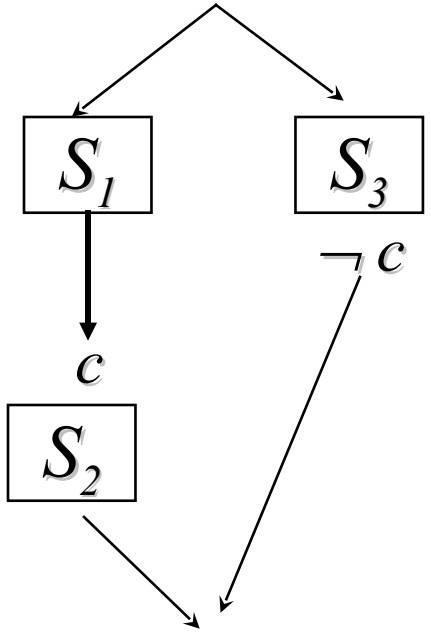
Planejamento Parcial - Exemplo



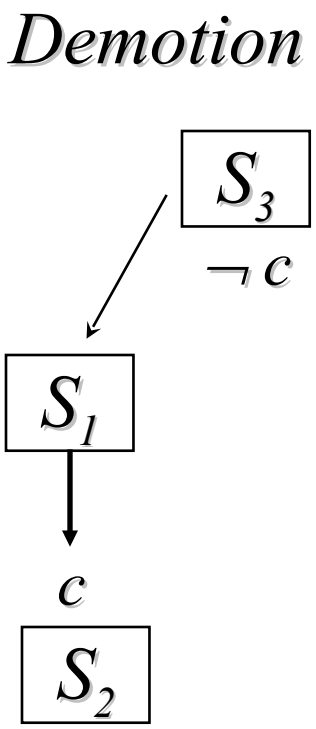
Problema da ameaça

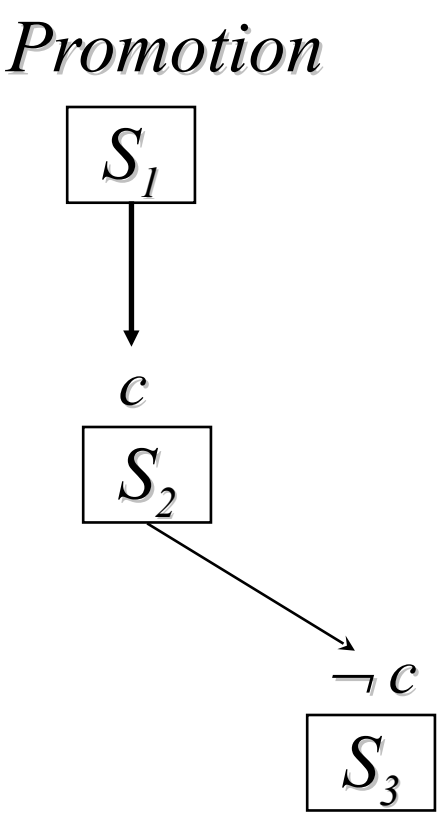
- **Ameaça**
  - ocorre quando os efeitos de um passo põem em risco as pré-condições de outro
- **Com testar?**
  - O novo passo é inconsistente com condição protegida
  - O passo antigo é inconsistente com nova condição protegida

S3 ameaça a condição c estabelecida por de S1 e protegida pelo link causal S1 para S2

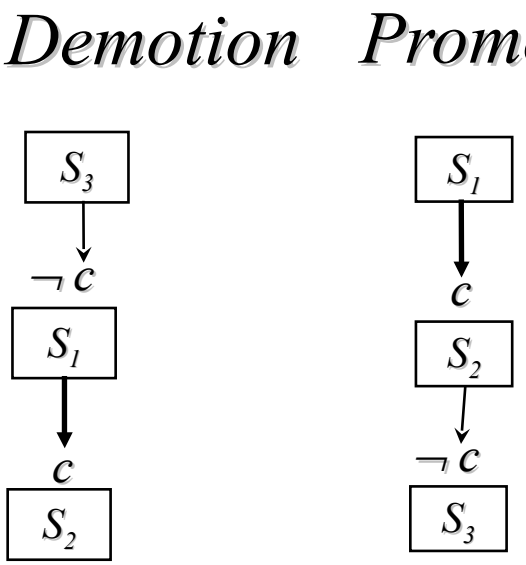


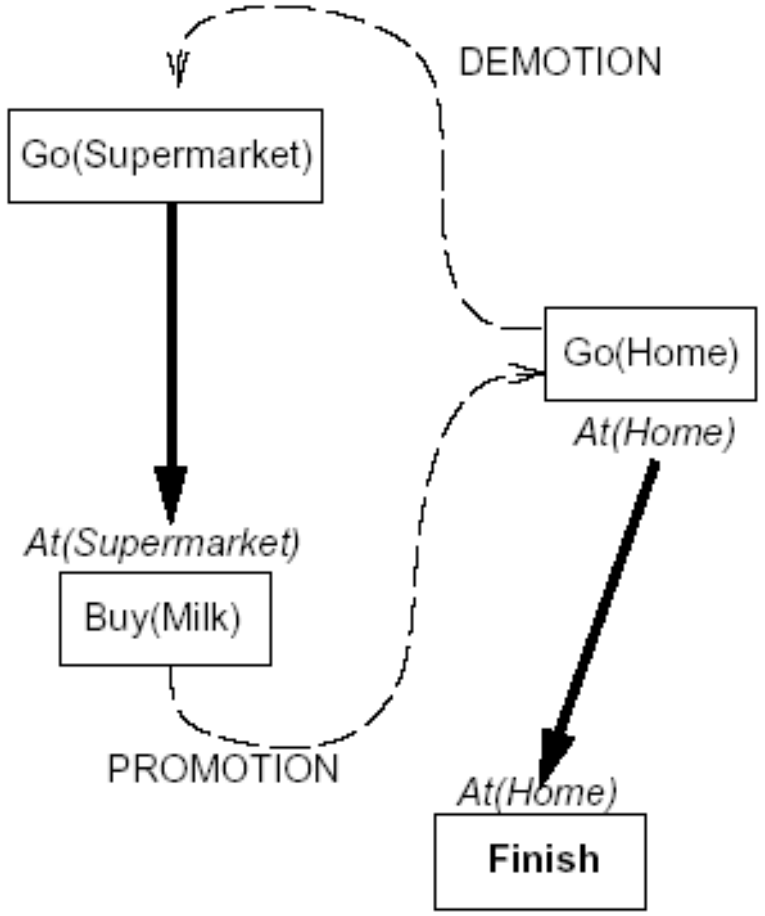
Ameaça - soluções



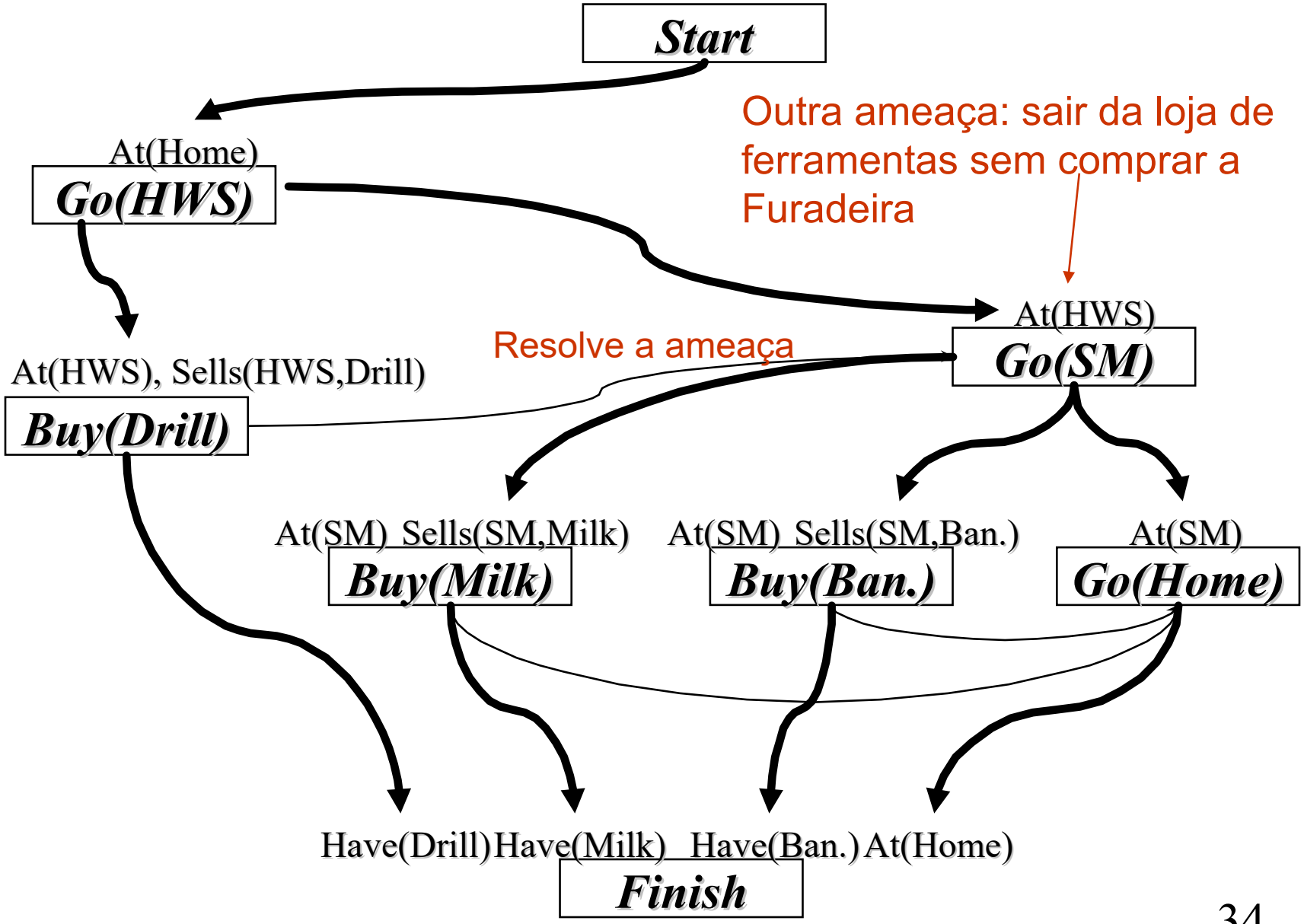


Ameaça - soluções

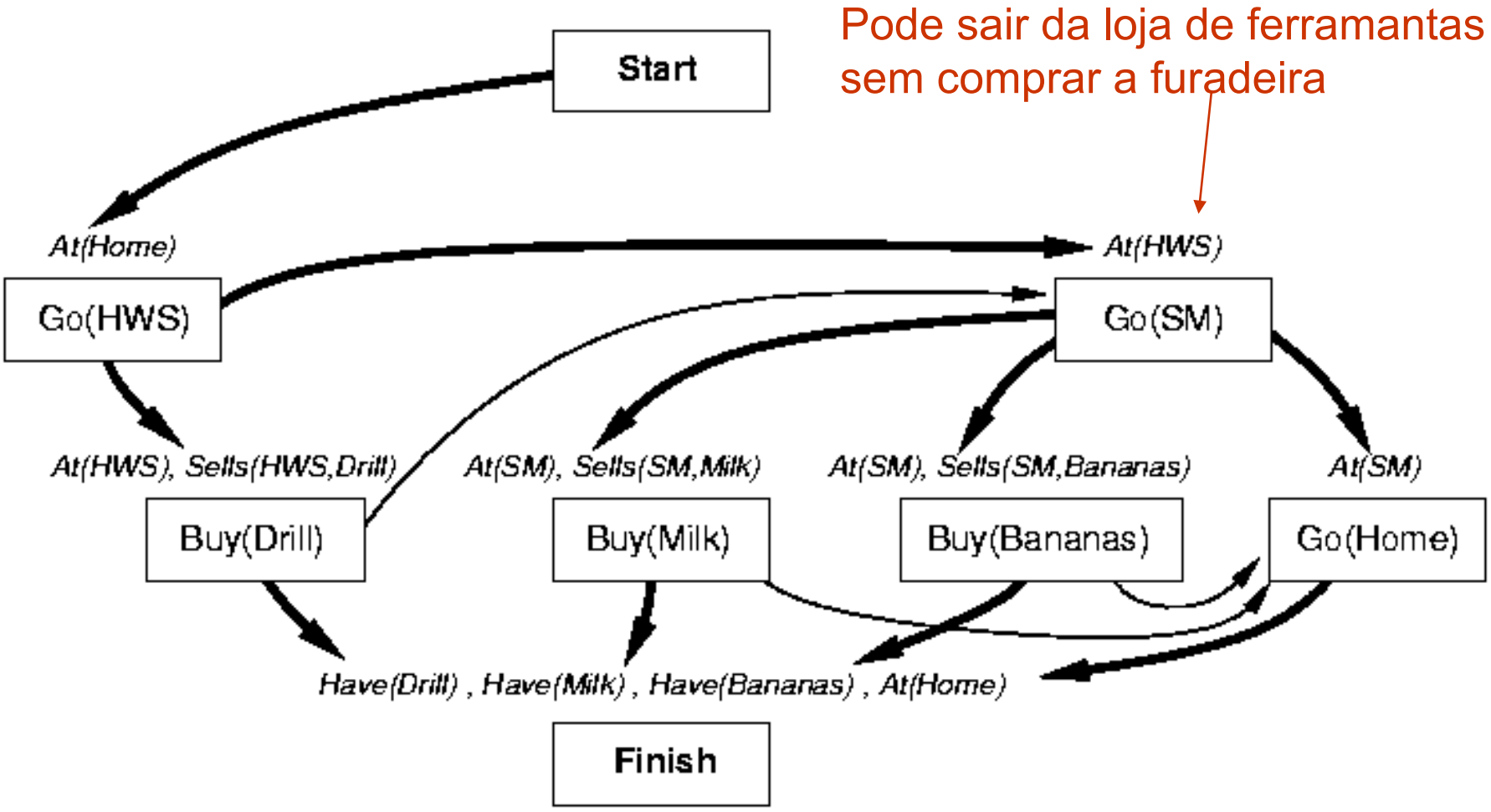




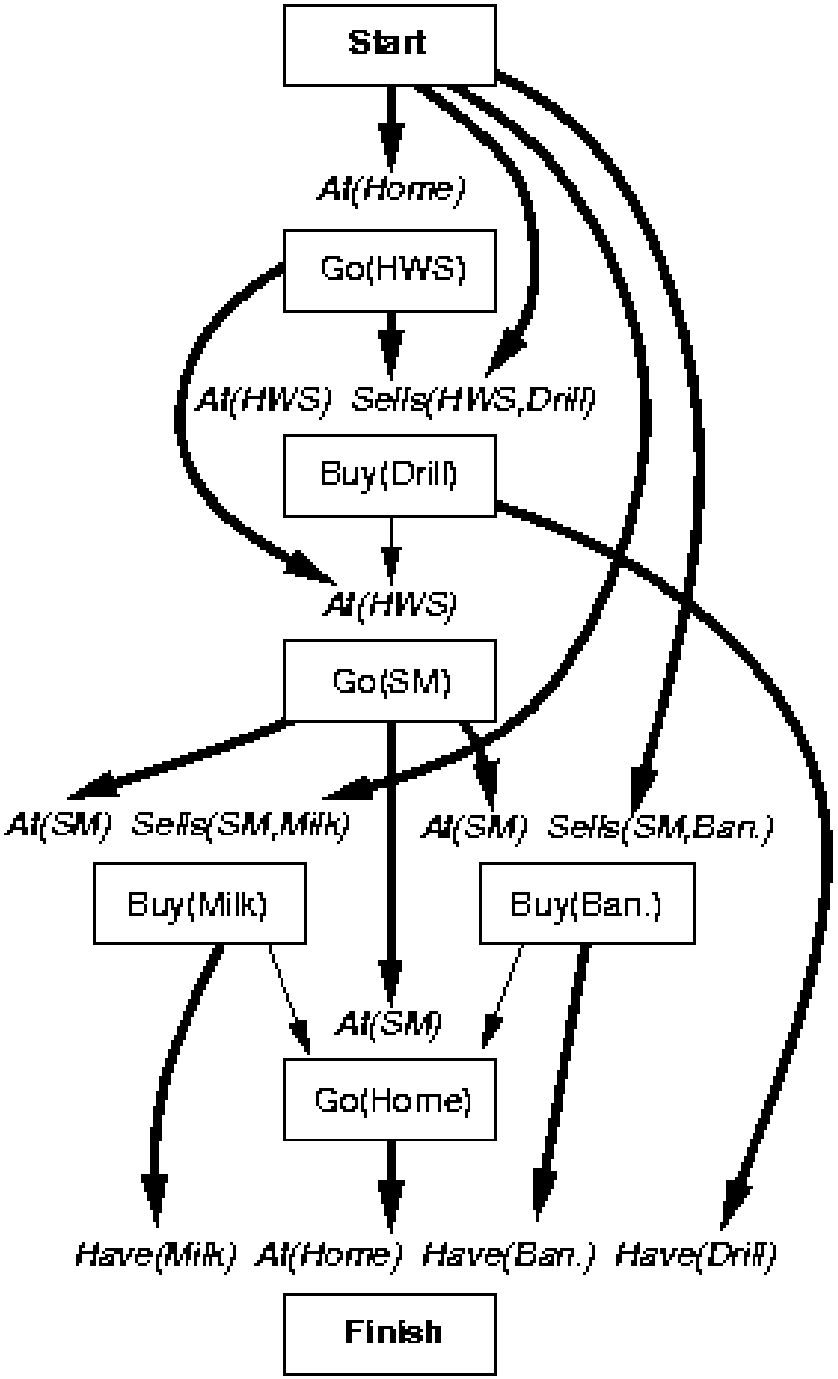
Planejamento Parcial - Exemplo



Planejamento Parcial - Exemplo



Planejamento Parcial - Exemplo

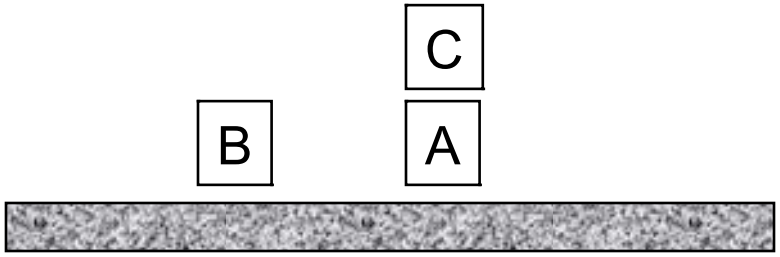


Engenharia do conhecimento

- **Decidir sobre o que falar**
- **Decidir sobre um vocabulário de condições, operadores e objetos**
- **Codificar os operadores para o domínio**
- **Codificar uma descrição da instância do problema**
- **Colocar o problema para o planejador existente e obter os planos**

Mundo dos blocos

- **O que falar**
  - um conjunto de blocos sobre uma mesa a serem empilhados numa certa ordem
  - só se pode mover um bloco se não houver nada em cima dele
- **Vocabulário**
  - On(b,x) - bloco b está em cima de x
  - Move(b, x, y) - mover b de x para y



Mundo dos blocos

- **Problema 1**
  - como representar que não há nada sobre um bloco? Não podemos usar $\neg\exists x$ on(x,b) ou $\forall$ x $\neg$on(x,b)
  - Solução: Clear(x)

- **Operador**

  Op(ACTION: Move(b, x, y),
  PRECOND: On(b, x) ^ Clear(b) ^ Clear(y)
  EFFECT: On(b, y) ^ Clear(x) ^ $\neg$On(b, x) ^ $\neg$Clear(y))

Mundo dos Blocos

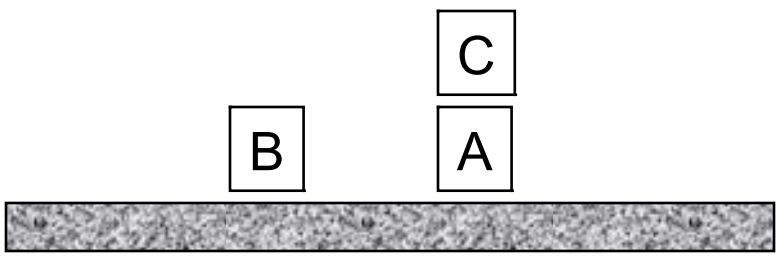
- **Problema 2: Clear(Table) !**
  - Cabem mais de um bloco sobre a mesa, logo...
  - Não é preciso testar clear(mesa) e nem modificar clear(mesa) quando novo bloco for posto em cima dela

- **Solução 2:**
  - Op(ACTION: MoveToTable(b, x),
    PRECOND: On(b, x) ^ Clear(b),
    EFFECT: On(b, Table) ^ Clear(x) ^ ¬On(b, x))
  - Redefinir o conceito de Clear(x) para “existe espaço livre em cima de x”

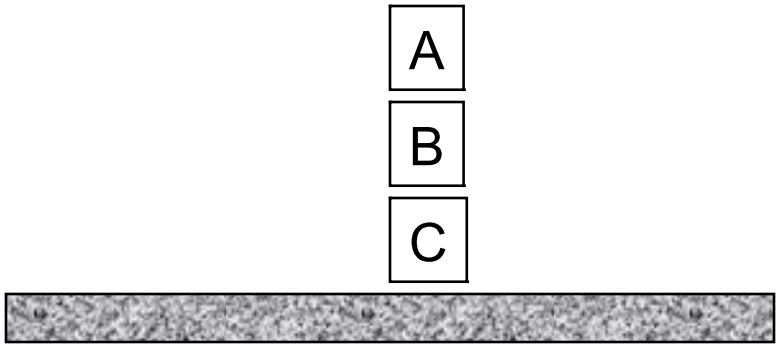
Mundo dos blocos

- **Com isto é possível resolver problemas do mundo dos blocos...**

Estado inicial



Estado final



Limitações de POP-STRIPS

- **Planejamento a um único nível de granularidade:**
  - de POP-STRIPS para POP-HAD-STRIPS
- **Precondições e efeitos não contextuais:**
  - de POP-STRIPS para POP-DUNC
- **Representação do tempo:**
  - de POP-STRIPS para POP-TI-STRIPS
- **Representação de recursos globalmente limitados:**
  - de POP-STRIPS para POP-RC-STRIPS
- **As aplicações reais mais complexas requerem:**
  - POP-TIRC-HAD-DUNC! :)

Aplicações

- **Construção de prédios:**
  - SIPE
- **Escalonamento de tarefas industriais**
  - TOSCA (Hitachi)
  - ISIS (Whestinghouse)
- **Construção, integração e verificação de espaçonaves:**
  - Optimum-AIV (Agência Espacial Europea)
- **Planejamento para Missões Espaciais**
  - Voyager, Telescópio espacial Hubble (NASA)
  - ERS-1 (Agência Espacial Europea)
- **etc.**